

POSIX vs. System V IPC: A Detailed Comparison with C Examples

Inter-Process Communication (IPC) mechanisms in Unix-like operating systems primarily follow two standards: **System V IPC** and **POSIX IPC**. These APIs allow processes to communicate via **pipes, message queues, shared memory, and semaphores**.

◆ Key Differences: POSIX vs. System V IPC

Feature	System V IPC	POSIX IPC
Naming	Identified by integer keys (ftok())	Uses path-like names (e.g., /shm_example)
Creation	msgget(), shmget(), semget()	mq_open(), shm_open(), sem_open()
Access Control	Uses permissions (IPC_CREAT, IPC_EXCL)	Uses file descriptor-style access
Cleanup	Requires manual removal (msgctl(), shmctl())	Automatically removed when last reference is closed
Portability	Older, still widely supported	Modern, preferred for new applications

1 Pipes

◆ System V: Anonymous Pipe

System V provides **anonymous pipes**, which are created using pipe(), allowing parent-child process communication.

Example: System V Pipe

```
#include <stdio.h>
#include <unistd.h>

int main() {
    int fd[2];
    pipe(fd);
    pid_t pid = fork();

    if (pid == 0) { // Child Process
        close(fd[1]);
        char buffer[20];
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd[0]);
    } else { // Parent Process
```

```

        close(fd[0]);
        write(fd[1], "Hello Child!", 12);
        close(fd[1]);
    }
    return 0;
}

```

◇ POSIX: Named Pipe (FIFO)

POSIX introduces **named pipes (FIFOs)** that persist beyond process termination.

Example: POSIX Named Pipe

```

#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

#define FIFO_PATH "/tmp/my_fifo"

int main() {
    mkfifo(FIFO_PATH, 0666); // Create named pipe

    pid_t pid = fork();
    if (pid == 0) { // Child Process
        int fd = open(FIFO_PATH, O_RDONLY);
        char buffer[100];
        read(fd, buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
        close(fd);
    } else { // Parent Process
        int fd = open(FIFO_PATH, O_WRONLY);
        write(fd, "Hello FIFO!", 11);
        close(fd);
    }
    return 0;
}

```

2 Message Queues

◇ System V Message Queue

System V queues are identified by integer keys and require explicit cleanup.

Example: System V Message Queue

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct msg {
    long type;
    char text[100];
};

int main() {
    key_t key = ftok("queuefile", 65);
    int msgid = msgget(key, 0666 | IPC_CREAT);
    struct msg message;

    message.type = 1;
    strcpy(message.text, "Hello System V IPC!");

    msgsnd(msgid, &message, sizeof(message), 0);
    printf("Message Sent\n");

    return 0;
}
```

◇ POSIX Message Queue

POSIX message queues are accessed via **file descriptor-like handles**.

Example: POSIX Message Queue

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <mqueue.h>

#define MQ_NAME "/posix_mq"

int main() {
    mqd_t mq = mq_open(MQ_NAME, O_CREAT | O_WRONLY, 0644, NULL);
    mq_send(mq, "Hello POSIX IPC!", 16, 0);
    mq_close(mq);
    return 0;
}
```

3 Shared Memory

◇ System V Shared Memory

Requires **explicit cleanup** (`shmctl()`).

Example: System V Shared Memory

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <string.h>

int main() {
    key_t key = ftok("shmfile", 65);
    int shmid = shmget(key, 1024, 0666 | IPC_CREAT);
    char *str = (char *)shmat(shmid, NULL, 0);

    strcpy(str, "Shared Memory Example");
    printf("Data written: %s\n", str);

    shmdt(str);
    return 0;
}
```

◇ POSIX Shared Memory

Cleaner interface using **file descriptor-style handles**.

Example: POSIX Shared Memory

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

#define SHM_NAME "/posix_shm"

int main() {
    int fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    ftruncate(fd, 1024);
    char *ptr = mmap(NULL, 1024, PROT_WRITE, MAP_SHARED, fd, 0);

    strcpy(ptr, "POSIX Shared Memory Example");
    printf("Data written: %s\n", ptr);

    munmap(ptr, 1024);
    close(fd);
}
```

```
    return 0;
}
```

4 Semaphores

◆ System V Semaphore

Uses **integer keys** and requires **explicit cleanup**.

Example: System V Semaphore

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/sem.h>

int main() {
    key_t key = ftok("semfile", 65);
    int semid = semget(key, 1, 0666 | IPC_CREAT);
    printf("Semaphore ID: %d\n", semid);
    return 0;
}
```

◆ POSIX Semaphore

Uses **named semaphores**.

Example: POSIX Semaphore

```
#include <stdio.h>
#include <fcntl.h>
#include <semaphore.h>

#define SEM_NAME "/posix_sem"

int main() {
    sem_t *sem = sem_open(SEM_NAME, O_CREAT, 0644, 1);
    printf("POSIX Semaphore created\n");
    sem_close(sem);
    return 0;
}
```

Summary: POSIX vs. System V IPC

IPC Type	System V API	POSIX API
----------	--------------	-----------

IPC Type	System V API	POSIX API
Pipes	pipe() (anonymous)	mkfifo() (named)
Message Queues	msgget(), msgsnd(), msgrcv()	mq_open(), mq_send(), mq_receive()
Shared Memory	shmget(), shmat(), shmdt()	shm_open(), mmap(), munmap()
Semaphores	semget(), semop(), semctl()	sem_open(), sem_wait(), sem_post()

◇ Which One to Use?

- Use **POSIX IPC** for **modern applications** and better portability.
- Use **System V IPC** for **legacy systems** requiring compatibility.